

DevOps Assignment

Server Hardening, Containerized Deployment, Reverse Proxy & Monitoring

Prepared by	Malay Joshi
Environment	Ubuntu 24.04 LTS (Cloud VM)
Scope	Task 1 — System hardening, Docker deployment, Nginx reverse proxy (TLS), monitoring stack
Sudo user	malay

1. Overview

This report documents the end-to-end execution of a DevOps assignment covering four areas: baseline system hardening and access control, containerized application deployment using Docker and Docker Compose, an Nginx reverse proxy with TLS termination, and a monitoring stack built on Prometheus and Grafana. All commands were executed on an Ubuntu 24.04 LTS host, with privileged operations run under the 'malay' account.

2. System Hardening & Access Control

2.1 System Update & Firewall Baseline

The system package index was refreshed and UFW (Uncomplicated Firewall) status was reviewed before applying any rule changes, and SSH access was explicitly allowed through the OpenSSH application profile.

```
sudo apt-get update
sudo ufw status verbose
sudo ufw allow OpenSSH
```

2.2 SSH Key Generation

An ed25519 SSH key pair was generated locally (with a passphrase) for authenticating to the server, replacing password-based SSH logins.

```
ssh-keygen -t ed25519 -C "malayace.joshi@gmail.com"
```

2.3 Deploy User Provisioning

A dedicated, least-privilege user (deploy-nginx) was created to own and operate the application deployment, separating deployment operations from the primary administrative account.

```
sudo adduser deploy-nginx
```

2.4 SSH Hardening

Cloud-init's default SSH drop-in was overridden and the main sshd configuration was edited to disable root login and password authentication, enforcing key-based access only.

```
sudo nano
/etc/ssh/sshd_config.d/50-
cloud-init.conf #
PasswordAuthentication no
sudo nano
/etc/ssh/sshd_config
```

PermitRootLogin	no
PubkeyAuthentication	yes
PasswordAuthentication	no
ChallengeResponseAuthentication	no
UsePAM	yes

2.5 Intrusion Prevention (Fail2Ban)

Fail2Ban was installed and enabled to automatically block brute-force SSH attempts, with a local jail configuration defining retry thresholds and ban duration for the sshd jail.

```
sudo apt update && sudo
apt install -y fail2ban
sudo systemctl enable --
now fail2ban
sudo cp
/etc/fail2ban/jail.conf
/etc/fail2ban/jail.local
sudo nano
/etc/fail2ban/jail.local
```

enabled	true
port	ssh
backend	systemd
maxretry	5
findtime	10m
bantime	1h

Verification confirmed the sshd jail active with a single running jail:

```
sudo fail2ban-client status
sudo systemctl restart fail2ban
```

2.6 Unattended Security Updates

The unattended-upgrades package was installed and configured to apply security patches automatically, remove unused dependencies, and avoid unattended reboots, keeping the host patched without manual intervention.

```
sudo apt update && sudo apt
install -y unattended-upgrades
```

```
sudo dpkg-reconfigure -plow
unattended-upgrades
sudo nano
/etc/apt/apt.conf.d/20auto-
upgrades
sudo nano
/etc/apt/apt.conf.d/50unattended-
upgrades
```

Update-Package-Lists	1
Unattended-Upgrade	1
Remove-Unused-Dependencies	true
Automatic-Reboot	false

Configuration was validated with a dry run, and the systemd timers confirmed as active and scheduled:

```
sudo unattended-upgrade --dry-run --debug
systemctl status unattended-upgrades
systemctl list-timers --all | grep apt
```

3. Containerized Application Deployment

3.1 Docker Engine Installation

Legacy Docker-related packages were removed, and Docker CE was installed from the official Docker APT repository using the GPG-signed package source, following Docker's recommended installation path for Ubuntu.

```
for pkg in docker.io docker-doc docker-compose podman-docker containerd runc; do
  sudo apt remove -y $pkg
done

sudo apt update && sudo apt install -y ca-certificates curl gnupg
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo "$VERSION_CODENAME")
stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-
compose-plugin
sudo systemctl enable --now docker
```

Installed versions were verified: Docker 29.6.1 and Docker Compose v5.3.1.

3.2 Rootless Docker Access for the Deploy User

The `deploy-nginx` user was added to the `docker` group so container operations do not require `sudo`, and access was validated with the `hello-world` test image.

```
sudo usermod -aG docker deploy-nginx
groups deploy-nginx
newgrp docker
docker run hello-world
```

3.3 Sample Application & Compose Stack

A minimal static HTML application was created and containerized behind the official `nginx` image, orchestrated with Docker Compose.

```
mkdir -p ~/sample-app && cd ~/sample-app
# index.html - "Docker Deployment Successful!"
docker compose up -d
```

The Compose service was initially exposed on port 8080 for verification. Since traffic is intended to be routed exclusively through the reverse proxy, the temporary UFW rule for port 8080 was removed once functionality was confirmed:

```
sudo ufw allow 8080/tcp # temporary, verification only
sudo ufw delete allow 8080/tcp
```

4. Reverse Proxy & TLS (Nginx)

Nginx was installed on the host and configured as a reverse proxy in front of the containerized application, terminating TLS at the proxy layer so the application container itself is not directly internet-facing.

```
sudo apt update && sudo apt install -y nginx
sudo systemctl enable --now nginx
```

4.1 Self-Signed Certificate

```
sudo mkdir -p /etc/nginx/ssl
sudo openssl req -x509 -nodes -days 365 \
  -newkey rsa:2048 \
  -keyout /etc/nginx/ssl/nginx.key \
  -out /etc/nginx/ssl/nginx.crt
# Common Name (CN): server public IP
```

4.2 Site Configuration

A dedicated server block was created for the application, the default Nginx site was disabled, and the configuration was syntax-checked prior to reload.

```
sudo nano /etc/nginx/sites-available/sample-app
sudo ln -s /etc/nginx/sites-available/sample-app /etc/nginx/sites-enabled/
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t
```

```
sudo systemctl restart nginx
```

4.3 Firewall & Verification

Only HTTP (80) and HTTPS (443) were opened at the firewall for public application traffic, in addition to SSH. The proxy and upstream container were verified locally before exposing the service externally.

```
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
curl http://127.0.0.1:8080
curl -k https://localhost
```

Final UFW policy confirms application traffic is exposed only through Nginx on ports 80/443, with SSH and the monitoring ports (3000, 3001, 9090) explicitly allowed for administrative access.

5. Monitoring Stack

A monitoring stack was deployed via an additional Docker Compose file comprising Prometheus, Grafana, Node Exporter, cAdvisor, and Uptime Kuma, giving host-level, container-level, and uptime visibility.

```
docker compose up -d
# Services: prometheus, grafana, node-exporter, cadvisor, uptime-kuma
```

5.1 Prometheus

Prometheus was configured to scrape the cAdvisor and Node Exporter endpoints; the Targets page confirms both jobs (cadvisor, node) reporting a healthy UP state.

5.2 Grafana

Grafana was provisioned with the Prometheus data source, and two community dashboards were imported: dashboard 193 (Docker/cAdvisor container metrics) and dashboard 1860 (Node Exporter host metrics), both rendering live CPU, memory, and uptime panels.

Grafana URL	http://<host>:3000
Username	admin
Dashboards imported	193 (cAdvisor), 1860 (Node Exporter Full)

6. Summary

The assignment delivered a hardened Ubuntu host (key-only SSH, Fail2Ban, automatic security patching, minimal firewall surface), a non-root deployment workflow via a dedicated service account, a containerized application fronted by an Nginx reverse proxy with TLS termination, and a full observability stack (Prometheus, Grafana, cAdvisor, Node Exporter, Uptime Kuma) for host and container-level metrics.

All configuration files referenced above (Nginx site config, Nginx SSL config, Docker Compose files, and Prometheus scrape config) are included as supporting attachments alongside this report.